

Pre-requisites : (For Before Entering Project) **PHASE - 0**

- ◆ 1. On EC2 (as `ubuntu` or `root` user), install the following system-wide:

Tool	Where to install it	Notes
Java (JDK)	Root user	Needed for Jenkins & Maven
Jenkins	Root user	Installed as a system service
Docker	Root user	Needed for Docker builds
Maven	Root user	Needed for Java project builds
AWS CLI	Root user	Needed to configure EKS access
kubectl	Root user	Needed to interact with EKS
<code>aws eks update-kubeconfig</code>	Run it as Jenkins user later	It generates the <code>~/.kube/config</code> for Jenkins
Helm	Root user	

After all the above installations, 🛠️ **You must 1st create the EKS cluster** (via Terraform, console, or CLI) In `ubuntu` user

- ◆ 2. Then switch to `jenkins` user:

```
sudo su - jenkins
```

Inside `jenkins` user:

- Check if `docker` , `kubectl` , `mvn` , `aws` commands are accessible.
- Run:

```
aws eks update-kubeconfig --region <your-region> --name <your-eks-cluster>
```

👉 This command **only updates or creates the kubeconfig file** to allow `kubectl` (and Jenkins) to **connect to an already existing EKS cluster**.

 [Read](#)

► This generates ``.kube/config`` file inside Jenkins user – so Jenkins can deploy to EKS.

✓ The ``--name <your-eks-cluster>`` part tells the AWS CLI which EKS cluster you want to connect to.

``--name dev-cluster`` → The name of your existing EKS cluster

Important point

In clear steps:

1. 🏗️ **You must 1st create the EKS cluster** (via Terraform, console, or CLI) In ubuntu user
2. 👤 Then, **log in as the jenkins user** on EC2.
3. 🛠️ Run this command to **configure** ``.kube/config`` file for Jenkins.
4. ✓ After that, `kubectl` (and Jenkins) can talk to the EKS cluster.

🛠️ Key Setup Notes:

- After Docker install:

```
sudo usermod -aG docker jenkins
```

Then reboot for Jenkins to use Docker.

- After AWS CLI install:
 - Either set up credentials (`aws configure`)
 - Or attach an IAM role with EKS permissions to the EC2

✓ Summary:

- Install **all tools system-wide** as root
- Then **make sure Jenkins user can use them** (especially Docker, AWS CLI, and `kubectl`)
- Finally, configure ``.kube/config`` **inside the Jenkins user**

✓ Step-by-Step Flow:

◆ 1. Login as `ubuntu` (or `root`) user on your EC2

This is your main setup user.

◆ 2. Install AWS CLI + `kubectl`

Do this globally so all users (including Jenkins) can access them.

◆ 3. Create your EKS cluster

You can do this using:

- **Terraform** (recommended for DevOps projects)
- or **`eksctl`**
- or AWS Console manually

➡ When you create it, AWS provisions the control plane and nodes.

◆ 4. Still as `ubuntu`, run:

```
export AWS_EKS_USE_LEGACY_AUTH=false
eksctl utils write-kubeconfig --region us-east-1 --cluster your-cluster-name
```

This command generate the `kubeconfig` **with** `client.authentication.k8s.io/v1beta1` directly using `eksctl` (and avoid outdated `v1alpha1`)

This confirms that your cluster is up and `kubectl` can talk to it.

✓ `export AWS_EKS_USE_LEGACY_AUTH=false`

- Purpose:
Forces AWS CLI and `eksctl` to **use the newer authentication method** based on `apiVersion: client.authentication.k8s.io/v1beta1`

✓ `eksctl utils write-kubeconfig --region us-east-1 --cluster your-cluster-name`

- Purpose:
Generates or updates the `~/.kube/config` file with **correct cluster access info** for

`kubectl` to communicate with your EKS cluster.

Then...

◆ 5. Switch to `jenkins` user:

```
sudo su - jenkins
```

◆ 6. Run the same `update-kubeconfig` command again:

```
export AWS_EKS_USE_LEGACY_AUTH=false
eksctl utils write-kubeconfig --region us-east-1 --cluster your-cluster-name
```

👉 This will generate `.kube/config` inside `/home/jenkins/.kube/config`, so Jenkins jobs can talk to the EKS cluster.

✅ Why this is needed:

- `ubuntu` user creates the cluster (has full setup access)
- `jenkins` user needs **only kubeconfig** to deploy workloads via pipeline

For your current DevOps projects and CI/CD pipelines using Jenkins, **choose EC2-based EKS (Managed Node Groups)** — not Fargate.

✅ `clusterconfig.yaml` — for EC2 Launch Type (Managed Node Group)

Clusterconfig

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: devops-eks-cluster
  region: ap-south-1
  version: "1.29"

nodeGroups:
  - name: devops-node-group
    instanceType: t3.medium
    desiredCapacity: 2
```

```
minSize: 1
maxSize: 3
volumeSize: 20
iam:
  withAddonPolicies:
    autoScaler: true
    cloudWatch: true
    ebs: true
```

Notes:

- ``name``: Your EKS cluster name will be ``devops-eks-cluster``
- ``region``: Change if you're in a different AWS region
- ``nodeGroups``: EC2-managed node group with 2 ``t3.medium`` nodes
- ``ssh``: Allows you to SSH into EC2 worker nodes (optional)
- IAM policies: Gives access to CloudWatch, EBS, and Cluster Autoscaler (best practice)

create cluster from this YAML :

```
eksctl create cluster -f clusterconfig.yaml
```
